

Documentación Técnica del Proyecto WEB FoodLoop

Desarrollo Web Integral



Universidad Tecnológica de Xicotepec de Juárez

Desarrollo Web Integral

Héctor Valderrábano González

Integrantes:

- MARTINEZ OTERO JOSUE ATLAI - 230166
- RAMIREZ REYES JONATHAN BALDERMAR - 230410
- RODRIGUEZ PEREZ CHRISTIAN PAÚL - 230809
- JOSE AGUSTÍN JIMENES CASTILLO - 230365

1. Introducción	4
2. Planteamiento del Problema.....	4
Desperdicio Orgánico y Económico	4
Falta de Creatividad Culinaria.....	4
3. Objetivos del Proyecto	4
Objetivo General	4
Objetivos Específicos	4
4. Caso de Estudio	5
Contexto	5
Solución Propuesta.....	5
5. Metodología Ágil: Scrum	5
Justificación de Scrum	5
6. Plan del Proceso de Desarrollo WEB	7
Fases del Ciclo de Desarrollo	8
1. Fase de Planificación y Diseño Analítico (Sprint 0)	8
2. Fase de Desarrollo de Infraestructura y Núcleo (Sprint 1).....	8
3. Fase de Gestión de Inventario Inteligente (Sprint 2)	8
4. Fase de Integración del Motor de IA (Sprint 3)	8
5. Fase de Estabilización, Reactividad y Pruebas (Sprint 4)	9
7. Arquitectura del Sistema	9
Justificación de la Arquitectura	9
Diagrama de Arquitectura — Vista General	10
Diagrama de Capas del Sistema	10
Vista de Componentes	11
Flujo de Interacción del Usuario(paul)	13
8. Propuesta de Patrones de Diseño	13
Patrón MVC (Model - View - Controller)	13
Patrón Middleware.....	14
9. Propuesta de Patrones de Diseño	14

Patrón Adapter (Adaptador).....	14
Patrón Observer	15
10. Justificación de Frameworks y Tecnologías	16
Tecnología.....	16
Tipo.....	16
Justificación en FoodLoop	16
11. Esquema de Pruebas	17
Tipos de Prueba (paul)	17
12. Configuración del Entorno de Desarrollo	17
Requisitos Mínimos del Sistema (paul)	17
Lista de Herramientas Utilizadas	18
Parámetros de Configuración	19
Variables de Entorno	22
Corto Plazo: Escaneo Automatizado de Insumos	25
Mediano Plazo: Integración de Hardware Domótico (IoT)	25
Largo Plazo: Conexión con Supermercados y Tiendas Locales	25
16. Conclusión	26

1. Introducción

La gestión del hogar y la alimentación diaria ha experimentado una desconexión notable debido al ritmo de vida actual. Gran cantidad de insumos y alimentos en los hogares terminan en la basura simplemente porque los usuarios olvidan lo que tienen en sus alacenas o carecen de ideas sobre cómo combinar los ingredientes disponibles antes de que caduquen.

Ante esta realidad surge **FoodLoop**: una plataforma web integral diseñada para funcionar como un recetario dinámico potenciado por Inteligencia Artificial. El sistema permite registrar el inventario de alimentos y, mediante algoritmos avanzados e integración con modelos de lenguaje, sugiere recetas personalizadas en tiempo real, minimizando el desperdicio y modernizando la planificación alimenticia.

2. Planteamiento del Problema

Los procesos manuales de control de insumos y la planificación de comidas en los hogares presentan diversas problemáticas críticas:

Desperdicio Orgánico y Económico

Los usuarios compran productos que ya tienen en existencia o permiten que alimentos perecederos caduquen por falta de visibilidad y monitoreo de las fechas de vencimiento.

Falta de Creatividad Culinaria

Ante ingredientes limitados o poco comunes en la despensa, las personas suelen descartar opciones de cocina complejas, cayendo en la monotonía alimentaria o recurriendo a gastos innecesarios en comida rápida.

3. Objetivos del Proyecto

Objetivo General

Desarrollar una plataforma web inteligente llamada **FoodLoop** que permita la gestión automatizada de inventarios de alimentos (despensa o sobras) y la generación dinámica de recetas mediante Inteligencia Artificial, optimizando el consumo doméstico.

Objetivos Específicos

- Diseñar una interfaz interactiva, responsiva y accesible utilizando React y Tailwind CSS.
- Implementar un módulo de gestión de despensa (CRUD) con alertas visuales de caducidad.
- Desarrollar una API REST robusta y segura utilizando FastAPI/Node.js para el procesamiento de peticiones.
- Integrar la API de Google Gemini para la interpretación y estructuración de recetas basadas en texto.

- Aplicar los patrones de diseño **Adapter** y **Observer** para aislar servicios externos y mantener la reactividad del sistema.

4. Caso de Estudio

Contexto

El análisis se centra en la optimización del consumo de alimentos en el ámbito doméstico y en pequeños comercios locales de cocina. Se identificó la necesidad urgente de automatizar el control de stock, especialmente de ingredientes de alta rotación u hortalizas con tiempos de vida cortos.

Solución Propuesta

FoodLoop centralizará el estado de la despensa. El usuario introduce lo que tiene disponible y el sistema calcula de inmediato recetas viables, priorizando los elementos más cercanos a vencer para "cerrar el ciclo" (el *loop*) del alimento.

5. Metodología Ágil: Scrum

La metodología seleccionada es Scrum, por su enfoque iterativo e incremental que permite entregas progresivas y adaptación continua ante cambios en los requerimientos.

Justificación de Scrum

- Entregas rápidas de funcionalidades completas por Sprint.
- Retroalimentación continua del Product Owner y usuarios.
- Flexibilidad para adaptar requerimientos durante el desarrollo.
- Organización colaborativa mediante roles bien definidos.
- Mejora continua a través de retrospectivas periódicas.
-

Roles del Equipo

Rol	Responsabilidades
Product Owner	Definir requerimientos, priorizar el Product Backlog, validar entregas y representar los intereses del negocio.
Scrum Master	Coordinar ceremonias Scrum, eliminar impedimentos, supervisar la metodología y facilitar la comunicación.
Equipo Dev	Programación, diseño de interfaces, pruebas, documentación e implementación de cada incremento.

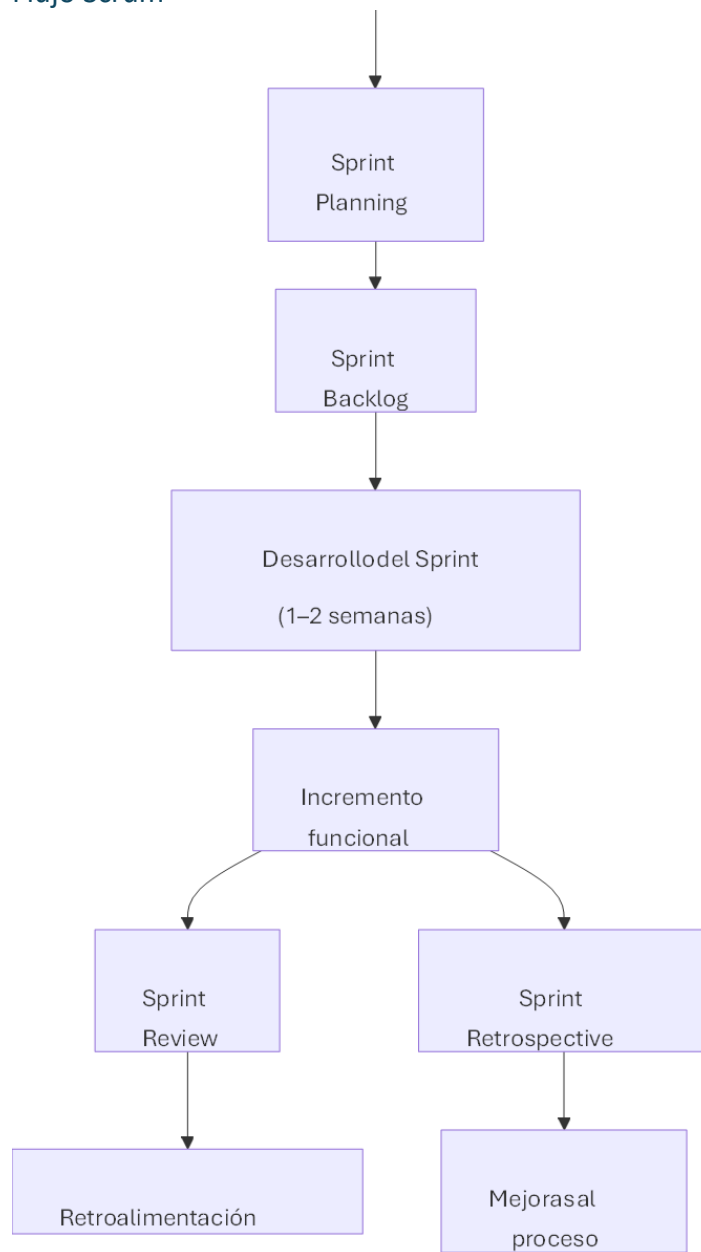
Ceremonias Scrum

Ceremonia	Descripción
Sprint Planning	Reunión al inicio de cada Sprint para seleccionar historias del Product Backlog y planificar actividades.
Daily Scrum	Reunión diaria (15 min) para sincronizar avances, identificar qué se hizo, qué falta y qué bloqueos existen.
Sprint Review	Presentación del incremento funcional al final de cada Sprint para obtener retroalimentación.
Sprint Retrospective	Análisis del proceso: qué salió bien, qué mejorar y compromisos de mejora para el siguiente Sprint.

Artefactos Scrum

Artefacto	Descripción
Product Backlog	Lista priorizada de todos los requerimientos del sistema.
Sprint Backlog	Subconjunto de historias seleccionadas para ejecutar en el Sprint actual.
Incremento	Resultado funcional y potencialmente entregable al finalizar cada Sprint.

Flujo Scrum



6. Plan del Proceso de Desarrollo WEB

El desarrollo de la plataforma **FoodLoop** se rige bajo un ciclo de vida incremental enfocado en la entrega continua de valor. El proceso se divide en una fase inicial de preparación (*Sprint 0*) destinada al modelado y diseño, seguida de **cuatro Sprints iterativos** de dos semanas cada uno, concluyendo con la fase de despliegue y liberación del sistema.

Fases del Ciclo de Desarrollo

1. Fase de Planificación y Diseño Analítico (Sprint 0)

- **Enfoque:** Configuración de bases conceptuales y diseño de arquitectura.
- **Actividades:** * Definición inicial del *Product Backlog* e historias de usuario primarias.
 - Diseño de los wireframes y el prototipo interactivo de la interfaz de usuario en Figma.
 - Diseño del modelo de datos relacional para PostgreSQL (Diagrama Entidad-Relación) y planeación del flujo de *prompts* para la IA.

2. Fase de Desarrollo de Infraestructura y Núcleo (Sprint 1)

- **Enfoque:** Configuración del entorno de desarrollo y cimientos del Backend.
- **Actividades:**
 - Inicialización del proyecto Frontend con React + Vite y configuración de Tailwind CSS.
 - Creación del servidor backend en FastAPI/Node.js y establecimiento de la conexión única mediante el patrón *Singleton* hacia PostgreSQL.
 - Desarrollo del módulo de seguridad y autenticación (Registro, Login, Hashing con bcrypt y generación de tokens JWT).

3. Fase de Gestión de Inventario Inteligente (Sprint 2)

- **Enfoque:** Implementación de la persistencia de datos y lógica de la despensa.
- **Actividades:**
 - Creación de las operaciones CRUD en el backend para la manipulación de ingredientes.
 - Desarrollo de las vistas y componentes reactivos en React (`MisIngredientes.tsx`) para agregar, editar y eliminar alimentos.
 - Implementación de lógica de negocio para el cálculo de caducidades y activación de alertas visuales por colores según la cercanía al vencimiento.

4. Fase de Integración del Motor de IA (Sprint 3)

- **Enfoque:** Conexión con servicios externos y procesamiento culinario automatizado.
- **Actividades:**

- Desarrollo del servicio conector hacia la API de Google Gemini implementando el patrón **Adapter**.
- Construcción e ingeniería de *prompts* para obligar a la IA a responder bajo estructuras estrictas de objetos JSON.
- Diseño de la vista del recetario dinámico en React, mostrando instrucciones paso a paso e ingredientes utilizados.

5. Fase de Estabilización, Reactividad y Pruebas (Sprint 4)

- **Enfoque:** Consistencia de datos, control de errores y aseguramiento de la calidad.
- **Actividades:**
 - Orquestación del manejo de estado global con React Context API bajo el patrón **Observer** para sincronizar la interfaz en tiempo real.
 - Implementación de la estrategia de respaldo (*Fallback*) en el *Adapter* por si la API de Gemini falla o agota su cuota.
 - Ejecución del esquema de pruebas funcionales, unitarias y de integración de endpoints mediante Postman.

7. Arquitectura del Sistema

Justificación de la Arquitectura

Se implementa una arquitectura desacoplada basada en el modelo **Cliente-Servidor mediante API REST**, permitiendo que el Frontend (React) y el Backend (FastAPI/Node.js) se comuniquen de forma asíncrona mediante JSON.

Criterio	Justificación en FoodLoop
Separación de Capas	Permite cambiar o modificar la interfaz de usuario en React sin alterar la lógica de generación de recetas por IA en el servidor.
Consumo Multicliente	La API REST expuesta permitirá en el futuro que un dispositivo IoT (como un refrigerador inteligente o báscula) actualice la despensa directamente.
Manejo de Respuestas Pesadas	Al procesar IA, el procesamiento pesado se queda en el servidor, aliviando la carga del navegador del usuario.

Diagrama de Arquitectura — Vista General

El diagrama de arquitectura general muestra cómo los principales componentes del sistema *FoodLoop* —usuario, frontend, backend, base de datos y servicios externos— se relacionan entre sí mediante una API REST. Esta vista permite entender el flujo de información y la interacción entre las distintas partes de la plataforma.

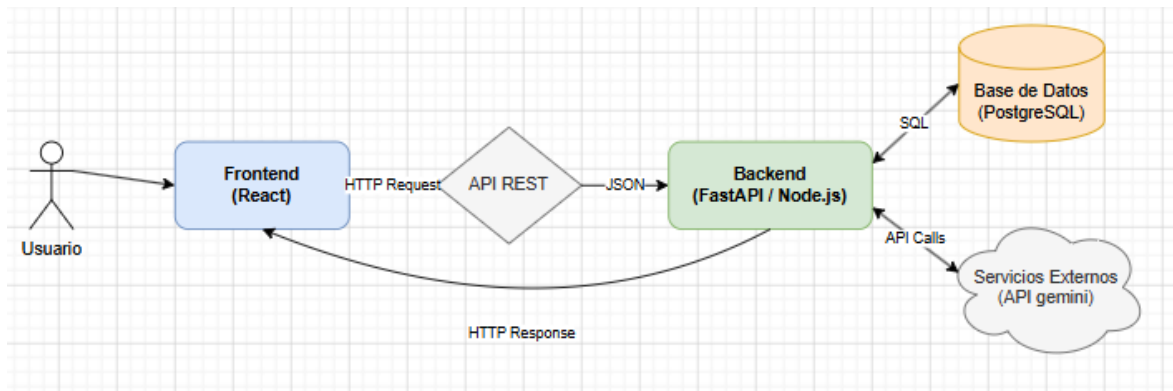
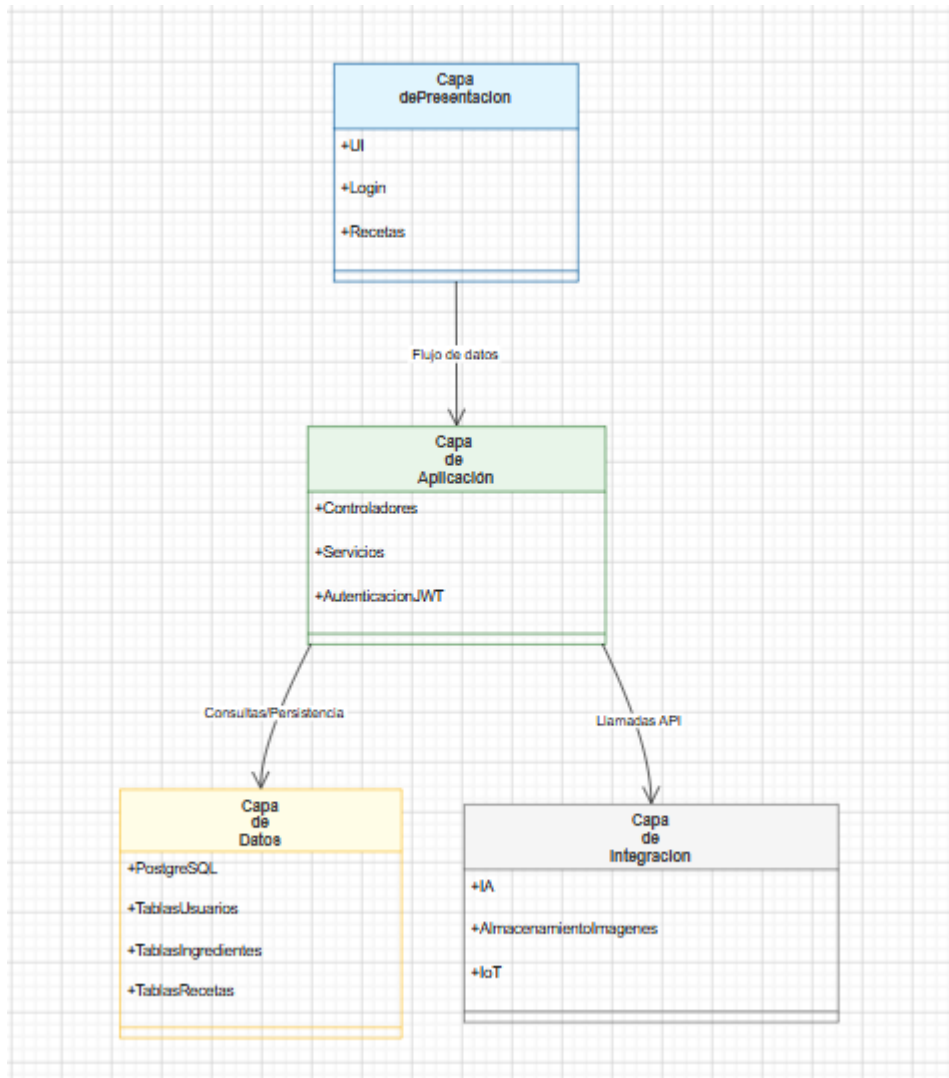


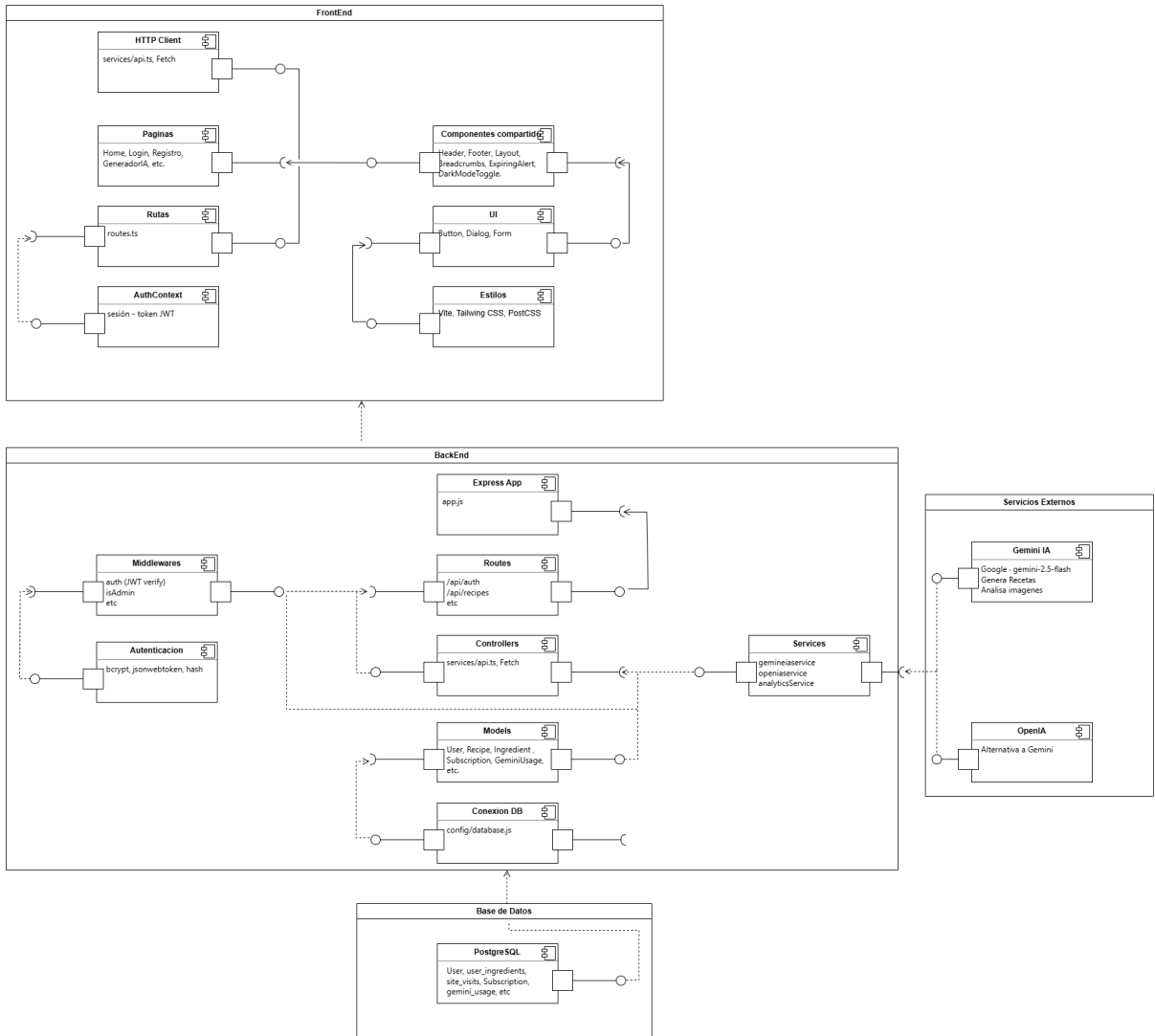
Diagrama de Capas del Sistema

El diagrama de capas organiza el sistema en niveles lógicos: presentación, aplicación, datos e integración. Esta representación facilita visualizar la separación de responsabilidades, asegurando modularidad, escalabilidad y facilidad de mantenimiento en el desarrollo de *FoodLoop*.

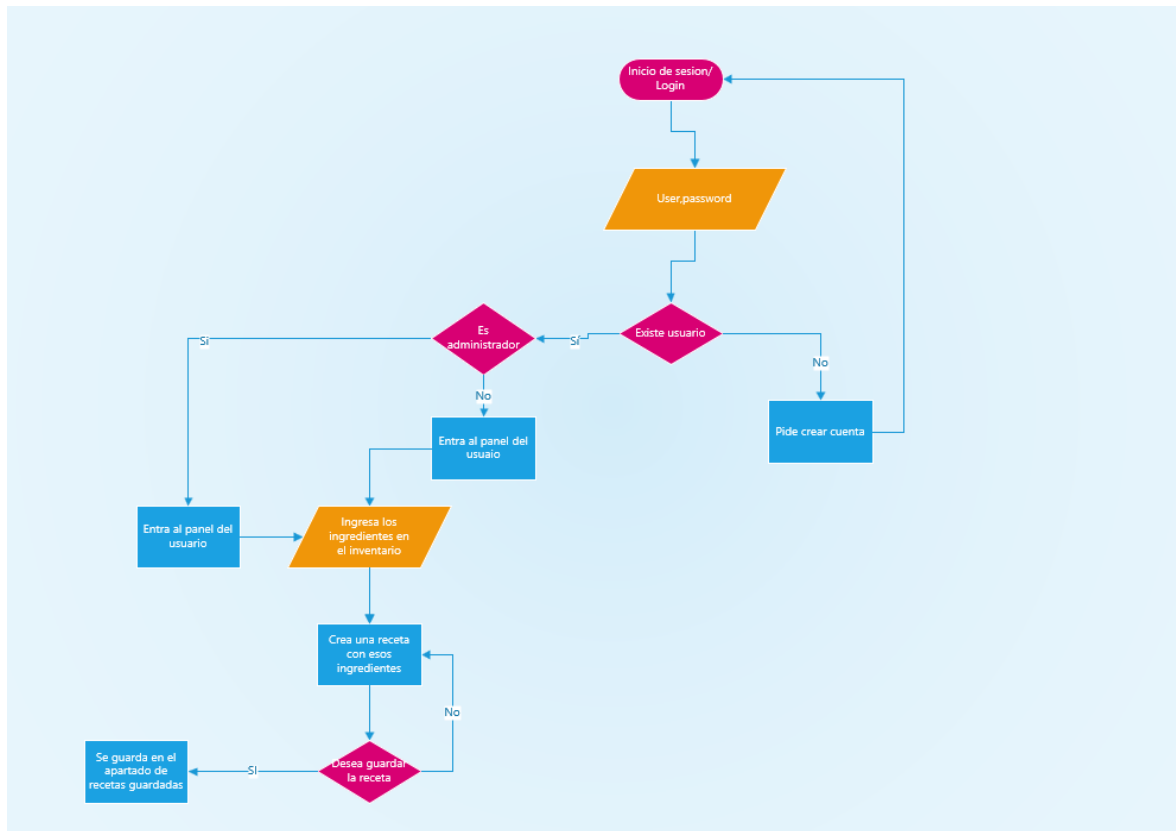


Vista de Componentes

La Vista de Componentes en UML sirve para representar la arquitectura de un sistema mostrando cómo se divide el software en módulos o componentes, como frontend, backend, base de datos, APIs o servicios externos, y cómo se comunican entre sí mediante interfaces ofrecidas y requeridas; esto ayuda a entender la estructura del sistema, organizar responsabilidades, visualizar dependencias, facilitar el mantenimiento y documentar la forma en que interactúan las diferentes partes de una aplicación.



Flujo de Interacción del Usuario(paul)



8. Propuesta de Patrones de Diseño

Patrón MVC (Model - View - Controller)

El patrón MVC es el principal patrón arquitectónico utilizado en el proyecto, ya que existe una separación clara entre la interfaz de usuario, la lógica del sistema y el acceso a los datos. Este patrón permite organizar la aplicación en diferentes capas con responsabilidades específicas, haciendo que el sistema sea más ordenado, escalable y fácil de mantener.

En el proyecto, la capa **View** corresponde al frontend desarrollado con React y Vite, donde se encuentran las páginas, componentes visuales y formularios que interactúan con el usuario. Esta parte del sistema únicamente se encarga de mostrar información y enviar solicitudes al backend.

La capa **Controller** está representada por los archivos dentro de la carpeta `controllers`, como `authController.js`, `recipeController.js` e `ingredientController.js`. Estos controladores reciben las peticiones HTTP provenientes del frontend, procesan la lógica necesaria y coordinan la comunicación entre las demás capas del sistema.

Finalmente, la capa **Model** corresponde a los archivos dentro de `models`, como `User.js`, `Recipe.js` e `Ingredient.js`. Estos modelos encapsulan toda la lógica relacionada con la base de datos, incluyendo consultas SQL, inserciones, actualizaciones y recuperación de información desde PostgreSQL.

La implementación del patrón MVC permite que cada componente del sistema tenga una responsabilidad específica, evitando mezclar lógica de negocio con la interfaz gráfica o con consultas a la base de datos. Gracias a esta separación, el mantenimiento del proyecto resulta más sencillo, ya que las modificaciones realizadas en una capa afectan mínimamente a las demás. Además, facilita el trabajo colaborativo entre desarrolladores y mejora la escalabilidad del sistema.

Patrón Middleware

El patrón Middleware se utiliza ampliamente en el backend desarrollado con Express. Este patrón consiste en utilizar componentes intermedios que interceptan y procesan las solicitudes HTTP antes de que lleguen a los controladores principales.

Cada middleware tiene una responsabilidad específica. Por ejemplo, `auth.js` verifica la validez del token JWT del usuario antes de permitir el acceso a ciertas rutas protegidas, mientras que `isAdmin.js` controla permisos administrativos y `validate.js` realiza validaciones sobre los datos enviados por el cliente.

El uso de este patrón permite reutilizar lógica importante en múltiples rutas sin duplicar código dentro de los controladores. Además, mejora significativamente la organización y seguridad del sistema, ya que separa procesos como autenticación, autorización y validación de la lógica principal de negocio.

Otra ventaja importante es que el patrón Middleware facilita la escalabilidad del backend. Nuevas validaciones o mecanismos de seguridad pueden añadirse fácilmente mediante nuevos middlewares sin modificar el funcionamiento interno de los controladores existentes.

9. Propuesta de Patrones de Diseño

Patrón Adapter (Adaptador)

El patrón Adapter se utiliza para permitir la comunicación entre componentes o sistemas que poseen interfaces diferentes. Su objetivo principal es actuar como un intermediario que adapta o traduce la forma en que dos sistemas interactúan, permitiendo que funcionen juntos, aunque internamente utilicen estructuras o métodos distintos.

En el proyecto, este patrón se implementa mediante la integración de servicios externos de inteligencia artificial como Gemini y OpenAI. Cada proveedor de IA utiliza APIs diferentes, con métodos propios de autenticación, formatos de solicitud y estructuras de respuesta distintas. Sin embargo, el sistema necesita utilizar ambos servicios de manera uniforme para generar recetas, analizar ingredientes y procesar información.

Los archivos `gemineiaservice.js` y `openaiservice.js` funcionan como adaptadores entre el backend y las APIs externas. Estos servicios encapsulan toda la lógica de comunicación con cada proveedor de inteligencia artificial y transforman sus respuestas en un formato comprensible y estandarizado para el sistema.

Gracias a esta implementación, los controladores del backend no necesitan conocer cómo funciona internamente cada API de inteligencia artificial, ya que únicamente interactúan con los servicios adaptadores. Esto reduce considerablemente el acoplamiento entre componentes y facilita el mantenimiento del sistema.

Además, el patrón Adapter permite integrar nuevos proveedores de inteligencia artificial en el futuro sin modificar la lógica principal de la aplicación. De esta manera, el proyecto obtiene una arquitectura más flexible, modular y escalable, capaz de adaptarse fácilmente a cambios tecnológicos o nuevas integraciones externas.

Patrón Observer

El patrón Observer puede identificarse principalmente en el frontend desarrollado con React, especialmente mediante el uso de Context API y manejo de estados globales.

Este patrón se basa en que múltiples componentes permanecen “observando” cambios en un estado compartido y reaccionan automáticamente cuando este cambia.

En el proyecto, el `AuthContext` funciona de manera similar a un observable. Cuando cambia el estado de autenticación del usuario, los componentes relacionados reaccionan automáticamente actualizando la interfaz.

Por ejemplo:

- al iniciar sesión,
- cerrar sesión,
- cambiar permisos,
- o actualizar información del usuario,

los componentes que dependen de ese contexto reciben automáticamente los cambios sin necesidad de recargar la página manualmente.

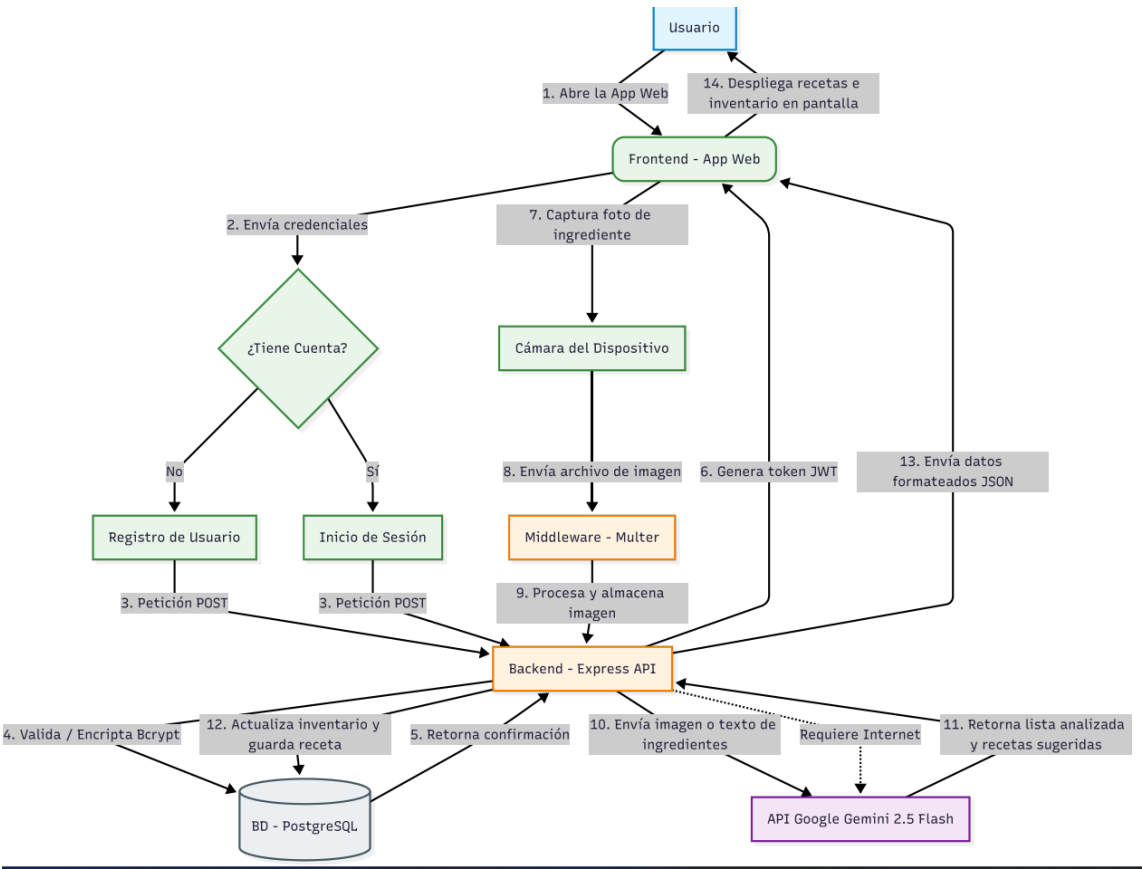
Esto mejora la comunicación entre componentes y permite mantener sincronizada la interfaz del sistema en tiempo real. Además, reduce considerablemente la complejidad al evitar el envío manual de datos entre múltiples componentes mediante props.

10. Justificación de Frameworks y Tecnologías

Tecnología	Tipo	Justificación en FoodLoop
React + Vite	Frontend	Entorno de desarrollo ultra rápido con renderizado basado en componentes eficientes y manejo de estado óptimo para interfaces dinámicas.
Tailwind CSS	Framework UI	Estilizado ágil mediante clases utilitarias que permite un diseño responsivo y moderno (módulos oscuros, alertas de color para alimentos por caducar).
FastAPI / Node.js	Backend	Procesamiento asíncrono de alto rendimiento, ideal para esperar las respuestas de la API de Gemini sin bloquear el servidor.
PostgreSQL	Base de Datos	Motor relacional robusto para asegurar la integridad de datos entre los usuarios, sus perfiles nutricionales y sus inventarios fijos.
Google Gemini API	Inteligencia Artificial	Modelo fundacional de lenguaje que procesa listas de cadenas de texto y devuelve combinaciones gastronómicas coherentes y creativas.

11. Esquema de Pruebas

Tipos de Prueba



12. Configuración del Entorno de Desarrollo

Requisitos Mínimos del Sistema (paul)

Requisito	Versión	Función
Node.js	V18.0.0 o superior	Entorno de ejecución para el servidor backend en JavaScript
PostgressSQL	V15.o o superior	Base de datos relacional para almacenar usuarios, inventario y recetas

Express	V5.2.1	Framework web para la creación de la API y manejo de rutas HTTP
Pg(node-postgres)	V8.20.0	Cliente que permite conectar y ejecutar consultas desde Node.js a PostgreSQL
@google/generative-ai	V0.24.1	SDK oficial para conectar con la API de Google Gemini 2.5 Flash y sugerir recetas
Multer	V2.1.1	Middleware para procesar y carga las fotos de los ingredientes tomadas por la cámara
Conexión a internet	Banda ancha (estable)	Requerida para consumir las APIs externas de Gemini, OpenAI y subir imágenes
Navegador Web	Versiones modernas	Interfaz de usuario compatible con HTML5/CSS3 y acceso a la cámara (MediaDevices API)
JSON WEB TOKEN (JWT)	V9.0.3	Manejo de la autenticación y sesiones seguras de los usuarios
Bcrypt	V6.0.0	Encriptación y hashing seguro de las contraseñas en la base de datos

Lista de Herramientas Utilizadas

Herramienta	Versión	Función en el Proyecto
Visual Studio Code	1.88+	Editor de código principal con extensiones para React, Python, Node.js y SQL Server/PostgreSQL.
Node.js	20 LTS	Entorno de ejecución para las dependencias del frontend (Vite) y controladores intermedios del backend.

Vite	5.x	Herramienta de construcción y scaffolding ultra rápida para la SPA de Frontend en React.
npm / pnpm	10.x	Gestor de paquetes para la instalación de dependencias del ecosistema JavaScript.
PostgreSQL	15.x / 16.x	Motor de base de datos relacional para el almacenamiento persistente de usuarios, ingredientes y favoritos.
Google Gemini API	vlbeta / v1	Servicio externo de inteligencia artificial fundacional para el procesamiento y estructuración de recetas.
Git	2.44+	Control de versiones distribuido del código fuente, manejando ramas de desarrollo (origin/development) y parches (bugfix).
GitHub	—	Repositorio remoto, control de versiones y hosting del código fuente del proyecto.
Postman / Thunder Client	10.x	Cliente HTTP para pruebas manuales de endpoints de la API de ingredientes y simulación de payloads de IA.
Figma	—	Diseño de interfaces de usuario (UI), mapas de navegación de la despensa inteligente y prototipado interactivo.

Parámetros de Configuración

FrontEnd — React (Vite)

Parámetro	Valor	Descripción
Puerto	5173	Puerto por defecto del servidor de desarrollo local de Vite.
Framework	React 18+	Librería basada en componentes y manejo modular del estado.

UI Framework	Tailwind CSS 3.x	Framework de CSS utilitario para estilizado ágil y diseño responsivo.
Lenguaje	TypeScript 5.x / JavaScript (ES6+)	Superset tipado para garantizar la consistencia en el tipado de recetas e ingredientes.
API Base URL	http://localhost:5000/api	Endpoint local para consumir las rutas del servidor backend.

BackEnd — API Service (FastAPI / Node.js)

Parámetro	Valor	Descripción
Puerto	5000	Puerto de escucha para el servidor backend local.
Runtime / Entorno	JavaScript / Node.js 20	Entorno de ejecución de la API de servicios lógicos.
Framework API	FastAPI / Express	Framework encargado de exponer los endpoints REST para la despensa y recetas.
Autenticación	JWT (expiración: 7 días)	Mecanismo de tokens firmados para asegurar las sesiones de los usuarios.
CORS	http://localhost:5173	Origen permitido en desarrollo (Servidor local de Vite).
Hashing	bcrypt (rounds: 12)	Algoritmo y factor de costo para el cifrado seguro de contraseñas de usuarios.

Base de Datos — Relacional (PostgreSQL)

Parámetro	Valor	Descripción
Puerto	5432	Puerto estándar por defecto de PostgreSQL.
Motor	PostgreSQL 15+	Sistema gestor de base de datos relacional para control estricto de inventarios.
Nombre de la BD	foodloop_db	Base de datos principal del ecosistema.
Tablas Clave	users, ingredients, saved_recipes	Estructuras relacionales de persistencia para el control transaccional.

Variables de Entorno

```
# Servidor y Entorno

PORT=5000

NODE_ENV=development


# Conexión Base de Datos Relacional (PostgreSQL)

DATABASE_URL=postgresql://postgres:password_seguro@localhost:5432/foodl
oop_db


# Integración de Inteligencia Artificial (Servicio Externo)

GEMINI_API_KEY=AlzaSyYourActualSecretKeyHere_FromGoogleAIStudio


# Seguridad y Tokenización

JWT_SECRET=clave_secreta_para_proteger_los_tokens_de_sesion_foodloop

JWT_EXPIRES_IN=7d


# Control de Accesos

CORS_ORIGIN=http://localhost:5173
```

Nota de Seguridad Importante: El archivo `.env` almacena credenciales y llaves privadas críticas (como el `GEMINI_API_KEY`). Debe ser excluido del control de versiones añadiendo explícitamente la línea `.env` al archivo `.gitignore` del proyecto.

Guía de Instalación Rápida

1. Clonar el repositorio del proyecto

```
git clone https://github.com/ChrisRodriguez-0430/FoodLoop.git
```

```
cd foodloop
```

2. Configurar e instalar dependencias del Backend

```
cd backend
```

```
npm install # (o configuración del entorno virtual si usas FastAPI)
```

```
cp .env.example .env
```

Modificar el archivo .env con tu clave real de Google AI Studio y credenciales de BD

3. Configurar e instalar dependencias del Frontend

```
cd ../frontend
```

```
npm install
```

4. Iniciar servicios en terminales independientes

Terminal 1 — Backend API:

```
cd backend && npm run dev
```

Terminal 2 — Frontend UI (Vite):

```
cd frontend && npm run dev
```

Acceder a la plataforma web en el navegador: <http://localhost:5173>

13. Seguridad del Sistema

Mecanismo	Implementación en FoodLoop
Autenticación JWT	Tokens firmados digitalmente con expiración de 7 días, validados en cabeceras HTTP mediante Middlewares antes de acceder a la información de la despensa.
Encriptación de Contraseñas	Hashing criptográfico mediante bcrypt con un factor de coste de 12; las contraseñas nunca se almacenan ni se comparan en texto plano en la base de datos.
Aislamiento de API de IA (Adapter)	El backend encapsula las llamadas a la API de Gemini. La clave de la API (GEMINI_API_KEY) nunca se expone al cliente frontend, evitando el robo de tokens de uso.
Protección de Rutas	Contextos globales y selectores de autenticación en React que actúan como protectores de rutas privadas en el Frontend, impidiendo accesos no autorizados a paneles de despensa.
Políticas CORS	Lista blanca estricta configurada en el servidor backend para rechazar de manera inmediata cualquier origen de petición ajeno al puerto local de desarrollo o al dominio de producción.
Variables de Entorno	Desacoplamiento total de credenciales técnicas, llaves de API e información de conexión a base de datos mediante archivos .env locales.

14. Riesgos del Proyecto

Riesgo	Probabilidad	Estrategia de Mitigación
Caídas u obsolescencia de la API de IA externa	Alta	Implementación del patrón de diseño Adapter con estrategias de respuesta de respaldo (Fallback) mediante almacenamiento local de recetas fijas en PostgreSQL si Gemini falla.

Respuestas incoherentes o "alucinaciones" de la IA	Media	Ingeniería de Prompts estructurada (Prompt Engineering) forzando de manera estricta a la API de Gemini a retornar únicamente formatos de datos JSON válidos y parseables.
Interrupción de la conectividad a Internet	Alta	Cacheo del inventario del usuario en el almacenamiento local del navegador para permitir la visualización de la despensa en modo offline.
Desafíos en la carga manual de ingredientes	Media	Diseño enfocado en la experiencia de usuario (UX) ágil con modales interactivos y categorías, sentando las bases para integración futura con hardware de escaneo (IoT).

15. Futuras Mejoras y Roadmap

Corto Plazo: Escaneo Automatizado de Insumos

Implementación de procesamiento de imágenes con visión por computadora (OCR) o lectura de códigos de barras para registrar los alimentos de la despensa de forma instantánea sin requerir tipado manual.

Mediano Plazo: Integración de Hardware Domótico (IoT)

Desarrollo y acoplamiento de sensores físicos de peso y proximidad en contenedores de cocina conectados a microcontroladores (como ESP32), permitiendo que la despensa web de **FoodLoop** se actualice de forma automatizada e inteligente en tiempo real según el consumo físico.

Largo Plazo: Conexión con Supermercados y Tiendas Locales

Alianzas con comercios de la región para permitir compras inteligentes directas desde la aplicación: si el sistema detecta que faltan ingredientes esenciales para tus recetas favoritas, genera automáticamente la lista de supermercado u optimiza la compra en un solo clic.

16. Conclusión

El proyecto **FoodLoop** representa una solución de software innovadora y sumamente necesaria para abordar problemas cotidianos críticos como el desperdicio alimentario doméstico y la ineficiencia en la gestión de insumos del hogar. La plataforma web trasciende la función de un recetario tradicional estático al transformarse en un ecosistema inteligente capaz de interactuar analíticamente con las existencias reales del usuario a través de Inteligencia Artificial.

La cuidadosa selección del stack tecnológico (**React**, **Vite**, **Tailwind CSS**, **FastAPI/Node.js**, **PostgreSQL**) provee al sistema de un entorno con un rendimiento óptimo, procesamiento asíncrono robusto para las llamadas a modelos de lenguaje y un almacenamiento de datos altamente seguro e íntegro. El desarrollo bajo la metodología ágil **Scrum** facilitó un proceso adaptativo e incremental, idóneo para absorber los retos técnicos surgientes durante la integración de herramientas de IA generativa.

Finalmente, la arquitectura implementada se ve fortalecida por la aplicación rigurosa de los patrones de diseño GoF elegidos: el **Adapter** garantiza la flexibilidad técnica del backend permitiendo cambiar, actualizar o mitigar fallos de la API de Google Gemini sin alterar el núcleo del software, mientras que el **Observer** dota al frontend de un comportamiento reactivo impecable, actualizando dinámicamente el entorno del usuario tan pronto como cambia su inventario. Con una visión escalable hacia el internet de las cosas (IoT), **FoodLoop** establece las bases tecnológicas idóneas para convertirse en un estándar en la automatización y sustentabilidad del hogar moderno.